

Assignment-12.3

NAME: V.rishikesh

HT.NO:2303A51386

Batch.No:06

TASK 1: Bubble Sort for Ranking Exam Scores

1. Prompt:

1. I want to sort student exam scores using Bubble Sort in Python.
2. The program should compare nearby scores step by step.
3. If one score is bigger, it should swap the values properly.
4. The sorting should stop early if the list is already sorted.
5. Also explain how fast or slow the algorithm works in general.

Code:

```
C: > Users > RISHIKESH > OneDrive > Documents > New folder > B.TECH > 3rd Year > 2nd Semster > AI assistant coding > lab.py > ...  
1  from typing import List  
2  def bubble_sort(scores: List[int]) -> None:  
3      n = len(scores)  
4      for i in range(n):  
5          swapped = False  
6          for j in range(0, n - i - 1):  
7              if scores[j] > scores[j + 1]:  
8                  scores[j], scores[j + 1] = scores[j + 1], scores[j]  
9                  swapped = True  
10         if not swapped:  
11             break  
12     def main() -> None:  
13         scores = [88, 55, 100, 73, 92, 85]  
14         print("Before sorting:", scores)  
15         bubble_sort(scores)  
16         print("After sorting:", scores)  
17         assert scores == sorted(scores), "Sorting failed"  
18     if __name__ == "__main__":  
19         main()
```

OUTPUT:

```
ts\New folder\B.TECH\3rd Year\2nd Semester\AI assistant coding'; & 'C:\Users\RISHIKESH\AppData\Local\Microsoft\WindowsApps\python3.11.exe' 'c:\Users\RISHIKESH\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '63037' '--' 'C:\Users\RISHIKESH\OneDrive\Documents\New folder\B.TECH\3rd Year\2nd Semester\AI assistant coding\lab.py'
Before sorting: [88, 55, 100, 73, 92, 85]
After sorting: [55, 73, 85, 88, 92, 100]
PS C:\Users\RISHIKESH\OneDrive\Documents\New folder\B.TECH\3rd Year\2nd Semester\AI assistant coding>
```

EXPLANATION:

- Bubble Sort compares two nearby values again and again.
- Bigger values slowly move to the end of the list.
- If no swapping happens, the loop stops early.
- It is slow for big data but fine for small lists.

TASK 2: Improving Sorting for Nearly Sorted

Attendance Records

1. Prompt:

- I have attendance roll numbers that are almost sorted.
- Only a few numbers are in the wrong position.
- Bubble Sort works but may not be the best choice here.
- I want a better algorithm suggested by AI.
- The program should show why it is faster.

2. CODE:

```
C:\> Users > RISHIKESH > OneDrive > Documents > New folder > B.TECH > 3rd Year > 2nd Semster > AI assistent coding > lab.py > ...
```

```
1 def insertion_sort(arr):
2     for i in range(1, len(arr)):
3         key = arr[i]
4         j = i - 1
5         while j >= 0 and key < arr[j]:
6             arr[j + 1] = arr[j]
7             j -= 1
8         arr[j + 1] = key
9     return arr
10
11 attendance = [1, 2, 4, 3, 5, 6]
12 print(insertion_sort(attendance))
13
```

3. OUTPUT:

```
ts\New folder\B.TECH\3rd Year\2nd Semester\AI assistant coding'; & 'C:\Users\RISHIKESH\AppData\Local\Microsoft\WindowsApps\python3.11.exe' 'C:\Users\RISHIKESH\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle\libs\debugpy\launcher' '54666' '--' 'C:\Users\RISHIKESH\OneDrive\Documents\New folder\B.TECH\3rd Year\2nd Semester\AI assistant coding\lab.py'
[1, 2, 3, 4, 5, 6]
PS C:\Users\RISHIKESH\OneDrive\Documents\New folder\B.TECH\3rd Year\2nd Semester\AI assistant coding>
```

4. Explanation:

- insertion sort places each element in its correct position.
- it works very fast when the list is almost sorted.
- fewer comparisons are needed compared to bubble sort.
- this makes it better for attendance systems.

TASK 3: Searching Student Records

1.PROMPT:

- I want to search student roll numbers from a list.
- Some data may not be sorted properly.
- I need one method for unsorted data.
- Another faster method should work for sorted data only.
- The program should show the difference clearly.

2.CODE:

```
lab.py
C: > Users > RISHIKESH > OneDrive > Documents > New folder > B.TECH > 3rd Year > 2nd Semester > AI assistant coding > lab.py > ...
1 def linear_search(arr, key):
2     for i in range(len(arr)):
3         if arr[i] == key:
4             return i
5     return -1
6
7 def binary_search(arr, key):
8     low, high = 0, len(arr) - 1
9     while low <= high:
10        mid = (low + high) // 2
11        if arr[mid] == key:
12            return mid
13        elif key < arr[mid]:
14            high = mid - 1
15        else:
16            low = mid + 1
17    return -1
18
19 students = [101, 104, 108, 110, 115]
20 print(binary_search(students, 110))
```

3.OUTPUT:

```
ts\New folder\B.TECH\3rd Year\2nd Semster\AI assistant coding'; & 'c:\Users\RISHIKESH\AppData\Local\Microsoft\WindowsApps\python3.11.exe' 'c:\Use
rs\RISHIKESH\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '60108' --- 'c:\Users\RISHIKESH\OneDrive\Do
cuments\New folder\B.TECH\3rd Year\2nd Semster\AI assistant coding\lab.py'
3
```

4.EXPLANATION:

- Linear Search checks each element one by one.
- Binary Search divides the list into halves.
- Binary Search works only on sorted lists.
- Binary Search is much faster for large data.

TASK 4: Quick Sort vs Merge Sort

1.PROMPT:

- I want to search student roll numbers from a list.
- Some data may not be sorted properly.
- I need one method for unsorted data.
- Another faster method should work for sorted data only.
- The program should show the difference clearly.

2.CODE:

```
C:\Users\RISHIKESH> OneDrive > Documents > New folder > B.TECH > 3rd Year > 2nd Semster > AI assistant coding > lab.py > ...
1 def quick_sort(arr):
2     if len(arr) <= 1:
3         return arr
4     pivot = arr[len(arr) // 2]
5     left = [x for x in arr if x < pivot]
6     middle = [x for x in arr if x == pivot]
7     right = [x for x in arr if x > pivot]
8     return quick_sort(left) + middle + quick_sort(right)
9
10 def merge_sort(arr):
11     if len(arr) <= 1:
12         return arr
13     mid = len(arr) // 2
14     left = merge_sort(arr[:mid])
15     right = merge_sort(arr[mid:])
16     return merge(left, right)
17
18 def merge(left, right):
19     result = []
20     i = j = 0
21     while i < len(left) and j < len(right):
22         if left[i] < right[j]:
23             result.append(left[i])
24             i += 1
25         else:
26             result.append(right[j])
27             j += 1
28     result.extend(left[i:])
29     result.extend(right[j:])
30     return result
```

3.OUTPUT:

```
31
32 # Static input
33 data = [45, 12, 78, 34, 89, 23]
34
35 print("Original Data:", data)
36 print("Quick Sort Output:", quick_sort(data))
37 print("Merge Sort Output:", merge_sort(data))
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python De

```
\WindowsApps\python3.11.exe' 'c:\Users\RISHIKESH\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '56311'
s\New folder\B.TECH\3rd Year\2nd Semster\AI assistant coding\lab.py'
Original Data: [45, 12, 78, 34, 89, 23]
Quick Sort Output: [12, 23, 34, 45, 78, 89]
Merge Sort Output: [12, 23, 34, 45, 78, 89]
PS C:\Users\RISHIKESH\OneDrive\Documents\New folder\B.TECH\3rd Year\2nd Semster\AI assistant coding> []
```

4.EXPLANATION:

- Quick Sort picks a pivot and splits the list.
- Merge Sort divides the list and merges it back.
- Merge Sort is stable and reliable.
- Quick Sort is faster in average cases.

Task 5: Optimizing Duplicate Detection Algorithm

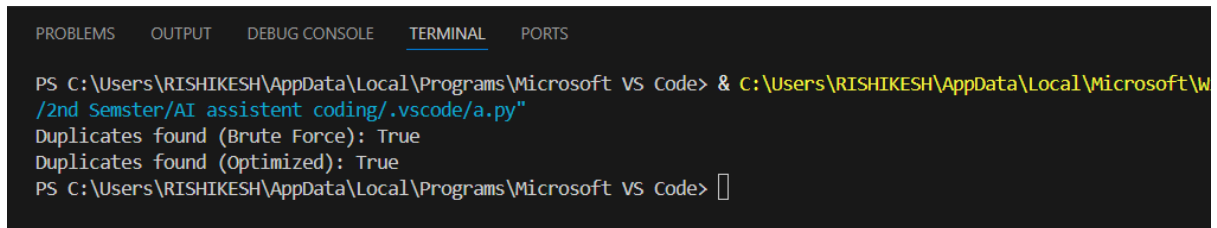
1.Prompt:

- Write a program to detect duplicate user IDs.
- Use a simple nested loop approach first.
- Analyze time complexity for large data.
- Optimize the program using a set.
- Compare performance of both approaches.

2.Code:

```
a.py
C: > Users > RISHIKESH > OneDrive > Documents > New folder > B.TECH > 3rd Year > 2nd Semster > AI assistant coding > .vscode > a.py > ...
1 from typing import Iterable
2 def has_duplicates_bruteforce(items: Iterable[int]) -> bool:
3     """Return True if there are duplicates using brute-force O(n^2) check."""
4     lst = list(items)
5     for i in range(len(lst)):
6         for j in range(i + 1, len(lst)):
7             if lst[i] == lst[j]:
8                 return True
9     return False
10 def has_duplicates_set(items: Iterable[int]) -> bool:
11     """Return True if there are duplicates using a set (O(n) expected)."""
12     seen = set()
13     for x in items:
14         if x in seen:
15             return True
16         seen.add(x)
17     return False
18 if __name__ == "__main__":
19     ids = [1, 2, 3, 2, 5]
20     print("Duplicates found (Brute Force):", has_duplicates_bruteforce(ids))
21     print("Duplicates found (Optimized):", has_duplicates_set(ids))
```

3.Output:



The screenshot shows a VS Code terminal window with the 'TERMINAL' tab selected. The terminal displays the following output:

```
PS C:\Users\RISHIKESH\AppData\Local\Programs\Microsoft VS Code> & C:\Users\RISHIKESH\AppData\Local\Microsoft\Windows\2nd Semster/AI assistant coding/.vscode/a.py"
Duplicates found (Brute Force): True
Duplicates found (Optimized): True
PS C:\Users\RISHIKESH\AppData\Local\Programs\Microsoft VS Code> █
```

5. Explanation:

- Brute force compares every element.
- Time complexity is very high.
- Set-based method checks faster.
- Optimized version is efficient for large data.